

Communication in Distributed Architecture

Optimizing Point-to-Point Data Transfer Throughput

Eddie Odiwuor Ogendero

March 26, 2026

Abstract

This research investigates the upper theoretical and practical limits of point-to-point data transfer throughput. While high-level frameworks like the Message Passing Interface (MPI) provide robust abstractions for distributed systems, they often introduce overhead through multiple memory-copy operations and context switching. We evaluate the impact of communication protocols, message sizes, and Modular Component Architecture (MCA) tuning, and propose a high-performance “Broadcaster” utilizing Linux `io_uring` and zero-copy splice/tee primitives. Our results demonstrate that bypassing the user-space buffer bottleneck allows data transfer rates to approach the physical saturation point of the underlying hardware interconnect

Contents

1	Introduction	3
2	High Performance Parallelism	3
2.1	Message Passing Interface (MPI)	3
2.1.1	The OpenMPI Architecture	4
2.1.2	Communication Protocols	4
2.2	Open Multi-Processing (OpenMP)	5
2.3	Parallel algorithm design	6
2.3.1	Task/Channel Model	6
3	Networking Foundation	7
3.1	Resumable / Partial Downloads	7
3.1.1	Range	7
3.1.2	Accept-Range	7
3.1.3	Content-Range	7
3.1.4	If-Range	8
3.2	Resumable / Multipart Uploads	8
3.2.1	Stages	8
3.2.2	Advantages	9
3.2.3	Best Practices	9
3.2.4	Standard Protocols	9
3.3	Latency	9
3.3.1	Calculations	10
3.4	Jitter	10
3.5	Data Processing Paradigm	10
3.5.1	Batch Processing	11
3.5.2	Stream Processing	12

4	Kernel-level Optimization	13
4.1	IO uring	13
4.2	Kernel TLS (KTLS)	13
5	Synthesis & Benchmarking	13
5.1	Broadcaster	13
6	Experimental Methodology	13
6.1	Hardware Setup	13
6.2	Software Environment	13
6.3	Software Environment	14
6.4	Benchmark Design	14
7	Optimization Strategies	14
7.1	Non-Blocking Communication	14
7.2	Remote Memory Access (RMA)	14
7.3	MCA Parameter Tuning	15
8	Results and Analysis	15
8.1	Bandwidth Analysis	15
8.2	Mathematical Performance Model	15
8.3	Comparison of Protocols	15
9	Conclusion	15

1 Introduction

The purpose of this research is to learn about the complexity of data movement problem in distributed systems.

Optimizing data transfer between two nodes involves minimizing the overhead introduced by the operating system, the network stack, and the physical limitations of the hardware.

The research will be structured in terms of these core concepts

- **High Performance Parallelism:** Message Passing Interface, point-to-point vs Collective communication.
- **Networking Foundations:** TCP/IP stack, Network fallacies, Chunking and Retries.
- **Kernel-level optimization:** io_uring, ktls (Kernel TLS) and zero-copy techniques.
- **Synthesis & Benchmarking:** Building the Broadcaster and measuring throughput vs latency.

2 High Performance Parallelism

2.1 Message Passing Interface (MPI)

Industry standard specification used for communication between processes in parallel computing

MPI attempts to solve the following problems:

- **Portability:**
Each vendor had propriety library for message passing thus requiring rewrites for different use cases.

Solution - MPI provides a standard API provided an MPI is installed (OpenMPI, MPICH)

- **Memory wall (*Distributed Memory Management*):**
Most large scale computers are distributed memory systems. Moving data between isolated memory manually is complex and error-prone.

Solution - MPI treats communication as explicit messages. MPI uses objects called *communicators* and *groups* to define which collection of processes may communicate with each other.

Provides functions that handle networking, buffering and memory copying.

- **Sender:** A process calls *MPI_Send()* (or a variant) to transmit a message.
- **Receiver:** A process calls *MPI_Recv()* to accept the message.
- **Message Envelope:** Data buffer (address, type, count), Source/destination ranks, Message tag, Communicator (*MPI_COMM_WORLD* by default)

- **Complexity of collective operations:**
Parallel algorithms often send pieces of data or collect results from multiple end points. Writing *one-to-many* or *many-to-one* patterns is tedious and usually inefficient.

Solution - MPI provides collective communication through built in functions.

- **Broadcaster** - sends data from one process to all.

- **Scatter/Gather** - Splitting a large vector into chunks for different processes or reassembling them.
- **Reduction** - performing calculations across all processors and return a single result.
- **Hardware heterogeneity:**
Computing clusters consists of different machines with different architecture e.g *big/little endian*.
Solution - MPI uses opaque datatypes and handles translation between different machines e.g. *(4, MPI_INT)* a vector with 4 integer values

2.1.1 The OpenMPI Architecture

OpenMPI is based on the Modular Component Architecture is a system that breaks down a software into independent, interchangeable modules, each encapsulating specific functionality. It is a series of projects, frameworks, components, and modules that are assembled at run-time to create an *MPI* implementation.

Each framework typically has one or more components that are used at run-time. **Point-to-point Management** Layer sits between the *MPI* API and the lower-level network transport layers. It manages message matching, ordering, and delivery guarantees required by the *MPI* standard. It decides how and through which transport a message will be sent. The **ucx** *pml* sends messages across **InfiniBand-based networks**.

The **btl** framework is used by the *MPI* layer to send bytes across different types underlying networks. The **tcp** *btl*, for example, sends messages across TCP-based networks

2.1.2 Communication Protocols

MPI provides three different communication methods that *MPI* processes can use to communicate with each other.

- **Point-to-Point Communication** - the most used communication method. It involves the transfer of a message from one process to a particular process in the same communicator. *MPI* provides blocking (*synchronous*) and non-blocking (*asynchronous*) point-to-point communication.
For the *blocking* communication, an *MPI* process *sends a message to another MPI process and waits until the receiving process completely and correctly receives the message* before it continues its work.
On the other hand, a sending process using *non-blocking* communication *sends a message to another MPI process and continues its work without waiting* to ensure that the message has been correctly received by the receiving process.
- **Collective Communication** - a process broadcasts a message to all processes in the same communicator including itself.
- **One-sided communication** - a process can directly access the memory space of another process without involving it.

```

1      /* Standard send in MPI. Behind the scenes, it will
2         issue a buffered send MPI_Bsend or a synchronous
3         send MPI_Ssend, based on whether the buffer attached
4         contains enough free space for the message. */
5      int MPI_Send(const void* buffer, int count,
6                   MPI_Datatype datatype, int recipient, int tag,
7                   MPI_Comm communicator);
8
9      /* Asynchronous blocking send ('B' == buffered); it
10         will block until a copy of the buffer passed is made
11         . MPI will send the buffer copied at a later point
12         */
13      int MPI_Bsend(const void* buffer, int count,
14                   MPI_Datatype datatype, int recipient, int tag,
15                   MPI_Comm communicator);
16
17      /* Synchronous blocking send; it will block until the
18         recipient has received the message. Ensure the
19         buffer and be used safely once MPI_Ssend return */
20      int MPI_Ssend(const void* buffer, int count,
21                   MPI_Datatype datatype, int recipient, int tag,
22                   MPI_Comm communicator);
23
24      /* Standard blocking send ('R' == ready); it will block
25         until the recipient has received the message.
26         Requires the recipient to have issued the
27         corresponding receive routine (MPI_Recv or MPI_Irecv
28         ) before MPI_Rsend is invoked. */
29      int MPI_Rsend(const void* buffer, int count,
30                   MPI_Datatype datatype, int recipient, int tag,
31                   MPI_Comm communicator);

```

2.2 Open Multi-Processing (OpenMP)

An application programming interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It allows developers to write parallel code easily and efficiently by adding simple compiler directives to their existing code.

MPI	OpenMP
Supports parallel computation for distributed-memory and shared-memory systems.	Supports parallel computation for shared-memory systems only.
A process-based parallelism.	A thread-based parallelism.
Each process has its own memory space and executes independently from the other processes.	Threads share the same resources and access shared memory.
Processes exchange data by passing messages to each other.	There is no notion of message-passing. Threads access shared memory.
Process creation overhead occurs one time.	It depends on the implementation. More overhead can occur when creating threads to join a task.

Table 1: Comparison between MPI and OpenMP

Utilizes the **Pragma** directive to allow the compiler to run the code sequentially if it doesn't support *OpenMP*.

1

```
#pragma omp [construct] [clause]
```

2.3 Parallel algorithm design

Methodology is based on the *task/channel model*, it facilitates the development of efficient parallel programs running on distributed-memory computers.

2.3.1 Task/Channel Model

Represents computations as a set of tasks that may interact with each other by sending messages through channels. Tasks can send local data values to other tasks via output ports and receive data from other tasks via input ports. Receiving in the model is **synchronous** while sending is a **asynchronous** operation.

Channel - is a message queue that connects one task's output with another task's input port. Data values appear at the input port in the same order as which they were placed in the output port at the end of the channel.

Foster's design encourages development of scalable parallel algorithms by delaying machine-dependent considerations to the later steps. The design's four steps involves:

- Partitioning.
- Communication.
- Agglomeration.
- Mapping

3 Networking Foundation

3.1 Resumable / Partial Downloads

Partial or resumable downloads allow a client to retrieve only specific portions of a resource rather than downloading the entire file from the beginning. This is essential for large media files, software packages, and any scenario where network interruptions are common. To support partial downloads, the **server (sender)** must explicitly implement HTTP range semantics and advertise that it can return byte ranges on demand.

3.1.1 Range

The **Range** header specifies which segment(s) of a resource the client wants to retrieve. It enables efficient skipping, seeking, and resuming of downloads without re-fetching previously received data.

Relevant status codes:

- **206 Partial Content** — The server supports range requests and is returning the requested segment(s).
- **416 Range Not Satisfiable** — The requested range is invalid (e.g., out of bounds).
- **200 OK** — The server does not support range requests or has chosen to ignore the header.

The header can request one or multiple subranges, similar to how media players jump to different timestamps in audio or video streams.

```
1      Range: <unit>=<start-end>
2      Range: <unit>=<n->           // from byte n to the end
3      Range: <unit>=<-n>           // last n bytes of the file
4      Range: <unit>=<s-e>, <s-e>, <s-e> // multiple non-
5      contiguous ranges
      Range: <unit>=<s-e>, <-e>      // mixed range
      types
```

Servers may return ranges in sequential order even if the client requested them out of order, ensuring efficient streaming without buffering the entire response.

3.1.2 Accept-Range

The **Accept-Ranges** header indicates whether the server supports range requests and which unit it accepts (typically bytes).

```
1      Accept-Ranges: none
2      Accept-Ranges: bytes
```

If the server returns **none**, clients should fall back to full downloads.

3.1.3 Content-Range

The **Content-Range** header describes where the returned segment fits within the full resource. It is included in responses with status **206**.

```
1      Content-Range: <unit> <range>/<size> // e.g. bytes
2      0-499/2000
3      Content-Range: <unit> <range>/*      // size unknown
      Content-Range: <unit> */<size>      // 416: invalid
      range
```

This header allows clients to:

- reconstruct the full file from multiple partial responses,
- verify progress,
- detect mismatches or corruption.

3.1.4 If-Range

The **If-Range** header makes a range request conditional. If the resource has not changed since the client last downloaded it, the server returns **206**. If it has changed, the server returns **200** with the full resource, ensuring the client does not stitch incompatible segments together.

The header may use either:

- **Last-Modified** timestamp, or
- **ETag** value

but not both simultaneously.

This mechanism is crucial for safe resumable downloads, especially when files may be updated on the server.

3.2 Resumable / Multipart Uploads

Resumable or multipart uploads are designed to reliably transfer large files over unstable or bandwidth-limited networks. Instead of sending the entire file in a single request, the file is divided into smaller, manageable chunks that are uploaded independently. This approach improves reliability, reduces memory usage, and allows uploads to continue even after interruptions.

Resumability is achieved by tracking which chunks have been successfully acknowledged by the receiver. If a failure occurs—whether due to network instability, client crash, or server timeout—the sender retransmits only the missing chunks rather than restarting the entire upload.

3.2.1 Stages

Initiation The sender begins by issuing a **POST** request to the receiver to start an upload session. The receiver responds with a unique *upload-id* and allocates temporary storage for incoming parts. This identifier is used to associate all subsequent chunk uploads with the same file.

Part Upload The file is sliced into fixed-size or dynamically sized chunks. Each chunk is uploaded independently along with its *part-number* and the *upload-id*. Because each part is isolated, uploads can be parallelized and retried individually.

Finalization Once all parts have been successfully uploaded, the sender issues a completion request containing the list of all *Etags* (hashes or identifiers returned by the receiver for each part). The receiver verifies the list, stitches the chunks together in the correct order, performs integrity checks, and returns a final confirmation.

3.2.2 Advantages

Parallel Streams:

The sender can open multiple concurrent HTTP connections (typically 4-8) to upload different chunks simultaneously. This maximizes throughput and better utilizes available bandwidth.

Memory Efficiency:

Only small slices of the file need to be loaded into memory at any given time. This avoids the overhead of buffering large files and keeps RAM usage predictable and low.

Error Isolation:

Failures affect only the chunk being uploaded. Instead of restarting the entire transfer, the sender retries only the failed part, significantly reducing wasted time and bandwidth.

3.2.3 Best Practices

Exponential Backoff - When a chunk upload fails, the sender should not retry immediately. Instead, it waits progressively longer intervals (e.g., 1s → 2s → 4s) before retrying. This prevents unnecessary congestion and aligns with common network-friendly retry strategies.

MD5 Validation - Each part should include an MD5 checksum header. The receiver validates the checksum before acknowledging the upload, ensuring data integrity and preventing corrupted chunks from being accepted.

Cleanup Process - Unfinished or abandoned uploads can accumulate over time. Lifecycle rules should be configured to automatically delete incomplete uploads after a defined retention period.

Dynamic Chunking - Chunk size can be adapted based on network conditions, example:

- 5 MB chunks for mobile or unstable networks
- 25 MB chunks for high-speed fibre connections

This improves reliability and reduces the likelihood of chunk-level failures.

3.2.4 Standard Protocols

- AWS S3 Multipart Upload
- tus.io Resumable Upload Protocol

3.3 Latency

Time it takes for data to go from the origin to the destination. This delay encompasses various factors such as:

- **Processing delay** - time it takes for data to be processed by network devices like routers & switches
- **Queueing delay** - time spent in queue waiting to be processed by the to the destination host.
- **Transmission delays** - time it takes to push the packet's bits into the wire.

- **Propagation delays** - time the signal takes to propagate through the medium. It would be a signal through an optic cable.

It refers to network's responsiveness and perceived speed and it is measured in milliseconds. The lower the "latency time" in milliseconds the better

3.3.1 Calculations

$$\begin{aligned}
 \text{Packet size } (L) &= \text{Packet size} \\
 \text{Average rate of arrival } (a) &= \frac{\text{packets}}{\text{sec}} \\
 \text{Bandwidth } (R) &= \frac{b}{\text{sec}} \\
 \text{Traffic intensity} &= L \cdot \frac{a}{R} \\
 \text{Propagation delay } (d_{\text{prop}}) &= \frac{m}{s} \\
 \text{Transmission delay } (d_{\text{trans}}) &= \frac{L}{R} \\
 \rho &= \frac{1}{R} \\
 \text{Time of 1st bit arrival (ms)} &= d_{\text{prop}} + \rho
 \end{aligned}$$

3.4 Jitter

The measure of the variation of latency over time, measured in *milliseconds*. It is the measure of the variability in packet arrival times, which can be caused by:

- **Network congestion:** High traffic volumes causing packets to take longer paths or experience delays.
- **Routing Changes:** Dynamic changes in the path that data packets take across the network.
- **Variable Path Lengths:** Differences in the physical distance data packets must travel.
- **Equipment Limitations:** Older routers with limited processing power struggle to handle modern network traffic volumes. When a router's buffer fills up, it must decide which packets to prioritize and which to delay or drop entirely.

It affects the quality of real-time communication. If the jitter is high then it may cause packet loss, poor audio and video quality transmission, and data streaming interruptions. Thus, jitter is a significant objective to consider in the performance of networks serving real-time applications.

3.5 Data Processing Paradigm

Data processing plays a role in how efficiently and effectively a system handles varying loads.

CAP theorem: distributed systems can guarantee at most two out of three properties [Eric Brewer, 2000]

- **Consistency** - every read receives the most recent read or an error. All clients sees the same data at same time no matter which node they connect to. For this to work data written to one node must be instantly forwarded & replicated in all other nodes before a write is deemed successful.

Aspect	Latency	Jitter
Definition	Time taken for data to travel from source to destination	Variation in latency over time
Measurement	Typically measured in milliseconds (ms)	Also measured in milliseconds (ms)
Components	Includes processing delay, queuing delay, transmission delay, propagation delay, etc.	Reflects inconsistency in latency due to network fluctuations
Impact	Affects overall speed and responsiveness of network connections	Impacts consistency and stability of data transmission
Nature	Generally a static value	Dynamic and can fluctuate over time
Examples	50 ms latency means it takes 50 ms for data to travel from source to destination	Jitter of 5 ms means latency varies by up to 5 ms over time

Table 2: Comparison between Latency and Jitter

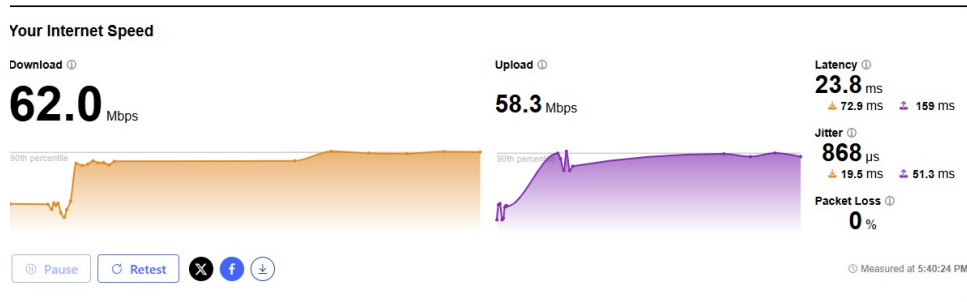


Figure 1: Jitter & latency speed test

- **Availability** - every request by a non-failing node must result in a response, without a guarantee it has the most recent version of data.
- **Partition tolerance** - system continues to operate despite an arbitrary number of messages being dropped/delayed by the network between nodes. In case of a *network partition* failure it must be decided whether:
 - Cancel the operation, decreasing availability but ensuring consistency.
 - Proceed with the operation, providing availability but risk inconsistency.

Network partition - division of a computer network into relatively independent subnet either by design to optimize them separately or due to failure of network devices. Distributed software should be partition-tolerant that is even after a network is partitioned, it should still work.

3.5.1 Batch Processing

Aggregation of data into discrete groups and processing them in a single scheduled operation, typically offline, triggered by time intervals (*e.g. nightly jobs*) or data thresholds (*e.g. 1GB patches*).

Ideal for:

- **Data warehousing** - central storage of data from multiple sources.
- **Extract Transform Load (ETL)** - collection of data from a source reshaping/processing it and moving it to an output.

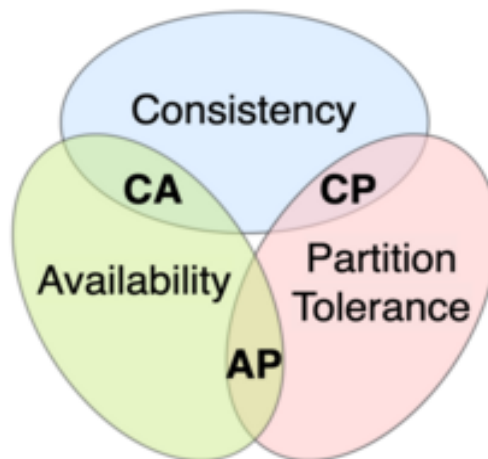


Figure 2: CAP theory properties

- Large scale reporting

There are a variety of *ETL* tools for batch processing: *Apache airflow*, *Hadoop*, *AWS Data pipeline*, *Oracle data integration*.

Core steps:

- **Data collection:**
Accumulation of data from sources into storage like *S3*, *HDFS*.
- **Processing:**
Execute computation/transformation on an entire batch.
- **Error handling:**
Retries on batch failures, with *idempotency* ensuring repeated runs don't duplicate outputs (*use of unique-ids like snowflakes for batch identifiers*)

$$\text{Throughput} = \frac{\text{batch size}}{\text{processing time}}$$

$$\text{Latency} = \text{Batch interval} + \text{processing time}$$

Resource utilization = Peak usage during processing (*e.g. 80% CPU for 30min*), idle otherwise.

Fault tolerance - use replication *e.g. 3X HDFS* and heartbeats for node liveness, failure trigger job restarts from checkpoints.

3.5.2 Stream Processing

Handle and analyse data in real time or near real time as they arrive. Data streams are then processed as they arrive, involving a series of operations such as filtering, transforming and data aggregation. Allowing real-time implementations like; live analytics, triggering alerts, real-time dashboards.

Core steps:

- **Data ingestion:**
Consume events from sources like message queues using *CDC* for database changes (*e.g. Kafka topics*)
- **Processing:**
Apply transformations using frameworks (*like Apache Flink or Kafka streams*).
- **Output:**
Emit results to sinks (*e.g. databases, caches*) in real-time.
- **Error handling:**
Idempotency for safe retries (*e.g. use unique_ids*), rate limiting (*e.g. token buckets*) to prevent overload and dead-letter queues for failed events.

Throughput = $N + \text{node throughput}$

Latency = ingress time + processing time + egress time

Resource utilization = For aggregation, window (*e.g. 5min*) affects memory (*1GB for 1M events/window*)

Aspect	Batch	Stream
Latency	High , data is collected over a period & processed in chunks at scheduled intervals.	Low , data is handled as it arrives.
Throughput	High for large batches (<i>e.g 1TB/hr</i>)	High for continuous flow (<i>e.g 1M events/sec</i>)
Resources	Peak during runs (<i>e.g 80% CPU for 1hr</i>).	Steady (<i>e.g 50% - 60% CPU continuously</i>).
Scalability	Vertical (<i>e.g larger machines</i>) or horizontal (<i>e.g spark clusters</i>)	Horizontal (<i>e.g Kafka partitions/brokers</i>)
Fault tolerance	Checkpoints (<i>e.g spark restart from checkpoints</i>)	Offsets & replications (<i>e.g Kafka resumes from last offset</i>).

Table 3: Comparison between batch and stream processing

4 Kernel-level Optimization

4.1 IO uring

4.2 Kernel TLS (KTLS)

5 Synthesis & Benchmarking

5.1 Broadcaster

6 Experimental Methodology

6.1 Hardware Setup

Describe the CPU, RAM, and NIC (Network Interface Card) of the two nodes.

6.2 Software Environment

The software stack was selected to ensure high-performance execution and minimal overhead during point-to-point data transfers. The environment is detailed as follows:

6.3 Software Environment

The software stack was selected to ensure high-performance execution and minimal overhead during point-to-point data transfers. The environment is detailed as follows:

- **Operating System:** The experiments were conducted on a Windows Subsystem for Linux running Ubuntu 24.04.3 LTS with the **Kernel version 6.6.87.2-microsoft-standard-WSL2**. To ensure maximum throughput, we verified that `sysctl` parameters for TCP/IP (such as `net.core.rmem_max` and `net.core.wmem_max`) were tuned to support large window sizes for high-bandwidth networks.
- **MPI Implementation:** We utilized **Open MPI v5.0.9**. The library was built against external **PMIx v5.0.9** and **PRRTE v3.0.12** dependencies to ensure a modern, modular process management layer. For intra-node communication within the WSL2 environment, the Modular Component Architecture (MCA) was verified to prioritize the `sm` (shared memory) and `self` Byte Transport Layers (BTL). This configuration enables low-latency, high-bandwidth data movement via shared memory regions, bypassing the network stack for local processes. Additionally, the `tcp` BTL was configured for management and coordination across the virtualized network interface.
- **Runtime Infrastructure:** To support the Open MPI 5.0 modular architecture, we deployed the **PMIx (Process Management Interface Exascale) v5.0.9** and **PRRTE (PMIx Reference RunTime Environment) v3.0.12** frameworks. These components were manually compiled and linked to provide a robust distributed process management layer, facilitating efficient process wire-up and synchronization during the `MPI_Init` phase.
- **Compiler and Flags:** All C source files were compiled using the **GCC 14.2.0** suite. To achieve "wire-speed" performance and ensure the CPU does not become the bottleneck during data packing/unpacking, the following optimization flags were applied:
 - `-O3`: Enables aggressive code optimizations including loop vectorization.
 - `-march=native`: Instructs the compiler to generate instructions specific to the host CPU architecture.
 - `-mtune=native`: Optimizes the generated code for the local processor's pipeline.
 - `-ffast-math`: Relaxes some strict IEEE floating-point compliance to further speed up arithmetic during data processing.

6.4 Benchmark Design

Cloudflare speedtest was used to measure the network download & upload speed, latency and jitter

7 Optimization Strategies

7.1 Non-Blocking Communication

Implementation of `MPI_Isend` and `MPI_Irecv` to hide latency.

7.2 Remote Memory Access (RMA)

One-sided communication using `MPI_Put` and `MPI_Get`.

7.3 MCA Parameter Tuning

Analysis of specific runtime flags like `btl_openib_eager_limit` and `pml_ucx`.

8 Results and Analysis

8.1 Bandwidth Analysis

Present the bandwidth curve relative to message size.

8.2 Mathematical Performance Model

The observed bandwidth $B(L)$ for a message of size L is modeled as:

$$B(L) = \frac{L}{\alpha + \beta L}$$

Where α represents latency and β represents the reciprocal of the peak bandwidth.

8.3 Comparison of Protocols

Graph comparing Blocking vs. Non-blocking vs. RMA.

9 Conclusion

Summary of the most effective configuration and final throughput achieved.

References

- [1] The Open MPI Project. *Open MPI: A High Performance Message Passing Library*. <https://www.open-mpi.org/>
- [2] MPI Basics by Tom Nurkkalla. Introduction to distributed computing with MPI. <https://www.youtube.com/>
- [3] Introduction to Parallel Programming using MPI. <https://arcca.github.io/intro-mpi/>
- [4] RookieHPC, learn about High-Performance Computing. <https://rookiehpc.org/>
- [5] Latency vs Jitter in Computer Networks. <https://www.geeksforgeeks.org/>
- [6] Cloudflare internet speed test. <https://speed.cloudflare.com/>
- [7] CAP theorem https://en.wikipedia.org/wiki/CAP_theorem
- [8] Batch vs Stream Processing: A comprehensive comparison of paradigm: <https://umamahesh.net/batch-vs-stream-processing-a-comprehensive-comparison-of-paradigms-with-use-cases/>
- [9] Batch vs Stream Processing: When to Use Each and Why It Matters. <https://www.datacamp.com/blog/batch-vs-stream-processing>
- [10] Parallel programming in C with MPI and openMP by Michael J. Quinn